Activation Functions, Weights & Full NN Training

From a pixel entering the network to the final prediction — step by step

BrainerX AI Masterclass · Session 2

What We'll Cover Today

A complete, clear deep dive — no skipping steps

01

What Are Weights?

The memory of the network — what they are, how they start, why they matter

02

Activation Functions

ReLU, Sigmoid, Softmax, Tanh — with graphs, math, and intuition

03

The Forward Pass

Step-by-step: image pixels → hidden layers → output

04

Loss & Backpropagation

How the network learns from its mistakes

05

Full Training Example

A digit classifier: image in → prediction out, every step

01 / What Are Weights?

The Simple Idea

A weight is just a number that controls how strongly one neuron influences the next.

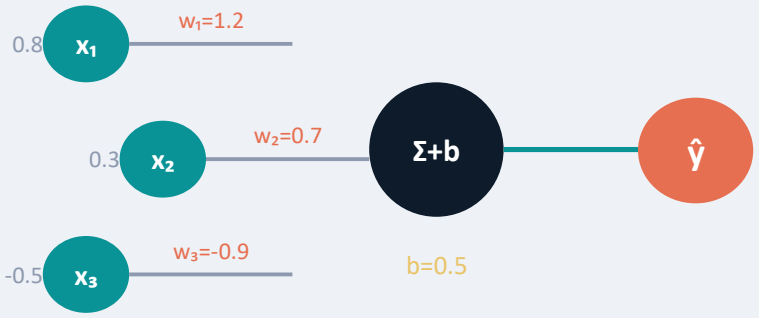
Think of it like a volume knob:

- Weight = 2.0 → this signal is amplified
- Weight = 0.0 → this signal is ignored
- Weight = -1.5 → this signal is inverted

Bias — The Offset

The bias is an extra number added so the neuron can fire even when all inputs are 0. It shifts the activation threshold.

$$\text{output} = w_1x_1 + w_2x_2 + w_3x_3 + b$$



Calculation Example

x_1=0.8, w_1=1.2 → contribution = 0.96

x_2=0.3, w_2=0.7 → contribution = 0.21

x_3=-0.5, w_3=-0.9 → contribution = 0.45

b=0.5 → z = 0.96+0.21+0.45+0.5 = 2.12

01 / How Weights Are Initialized

You can't start with all zeros — here's why, and what to do instead

All Zeros

Symmetry Problem

Every neuron computes the same output. Backprop gives the same gradient. All neurons learn the same thing → the network doesn't learn at all!

Random Small Values

Breaks Symmetry

Weights drawn from a normal distribution like N(0, 0.01). Each neuron starts different → they learn different features. But can still vanish with deep nets.

Xavier / Glorot Init

Best for Sigmoid/Tanh

$W \sim \text{Uniform}(-\sqrt{6/(n_{in}+n_{out})}, +\sqrt{6/(n_{in}+n_{out})})$

Scales by layer size to keep signal strength stable.

He Initialization

Best for ReLU

$W \sim N(0, \sqrt{2/n_{in}})$

Accounts for ReLU cutting half the values. Default in PyTorch Conv/Linear layers.

02 / Why Do We Need Activation Functions?

The Core Problem

Without activation functions, stacking layers of neurons is pointless — the whole network collapses into a single linear equation.

WITHOUT Activation

Layer 1: $z_1 = W_1x + b_1$

Layer 2: $z_2 = W_2z_1 + b_2$

Substituting:

$z_2 = W_2(W_1x + b_1) + b_2$

$= (W_2W_1)x + (W_2b_1+b_2)$

= Still just one linear equation!

No matter how many layers.

WITH Activation

Layer 1: $a_1 = \text{ReLU}(W_1x + b_1)$

Layer 2: $a_2 = \text{ReLU}(W_2a_1 + b_2)$

Non-linearity is introduced!

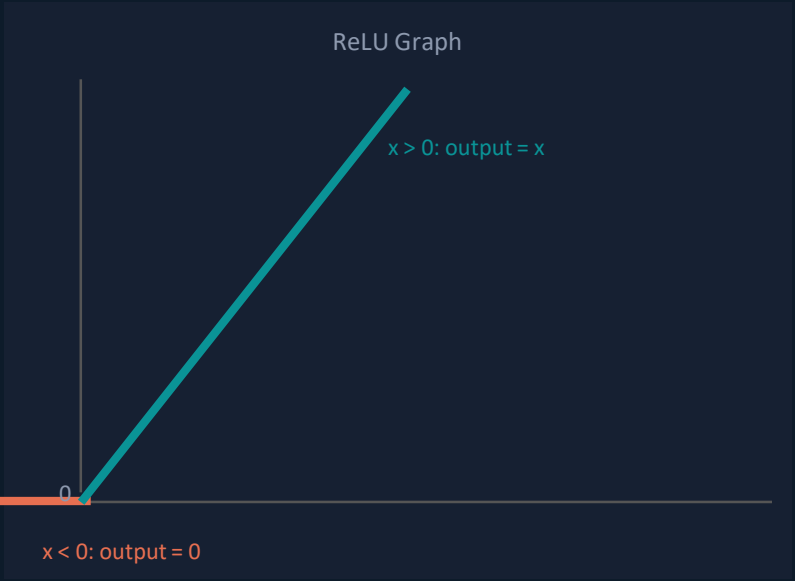
Each layer can now learn to detect curved boundaries, patterns, and complex shapes in data.

→ This is what makes deep learning powerful.

02 / ReLU — Rectified Linear Unit

The most widely used activation function in hidden layers

$$f(x) = \max(0, x)$$



Simple & Fast

Just one comparison: is x > 0? Extremely cheap to compute.

Solves Vanishing Gradient

Gradient is exactly 1 for positive values — doesn't shrink during backprop.

Sparse Activation

Negative neurons output 0 → only relevant neurons fire. Like biological neurons.

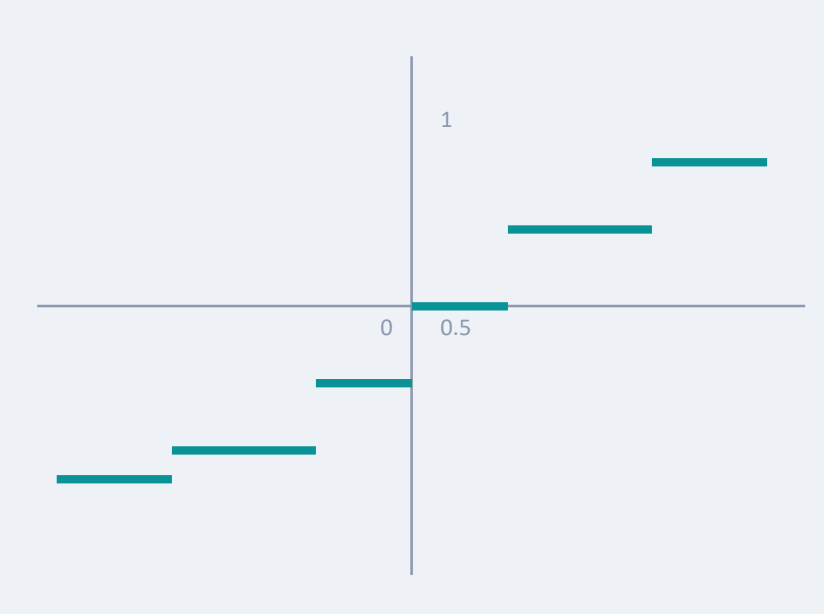
Dying ReLU Problem

If a neuron always gets negative input, its gradient = 0 forever. Solution:

PyTorch: torch.nn.ReLU() or F.relu(x)

02 / Sigmoid — Squash to (0, 1)

$$\sigma(x) = 1 / (1 + e^{-x})$$



When to Use Sigmoid

Output layer for binary classification

Is this email spam? Yes/No → output is a probability between 0 and 1.

Outputs a probability

$\sigma(-10)\approx 0$ · $\sigma(0)=0.5$ · $\sigma(10)\approx 1$

Smooth, differentiable everywhere.

Vanishing Gradient Problem

At extremes ($x \ll 0$ or $x \gg 0$), gradient ≈ 0.

Deep nets with sigmoid can stop learning. Use ReLU for hidden layers instead.

Not Zero-Centered

Outputs always positive (0 to 1), which can cause zig-zag weight updates. Tanh

PyTorch: torch.nn.Sigmoid() or torch.sigmoid(x)

02 / Softmax — Multi-Class Probabilities

Always used in the OUTPUT layer when there are 3+ classes

$$\text{Softmax}(x_i) = e^{x_i} / (e^{x_1} + e^{x_2} + \ldots + e^{x_n}) \quad \rightarrow \quad \text{All outputs sum to 1.0}$$

Concrete Example: Classifying digits 0–9

Raw Scores (logits)

Digit 3: 4.2

Digit 7: 2.1

Digit 1: 0.5

Others: ~-1

e^x then normalize

After Softmax (probabilities)

Digit 3: 0.86

Digit 7: 0.09

Digit 1: 0.03

Others: 0.02

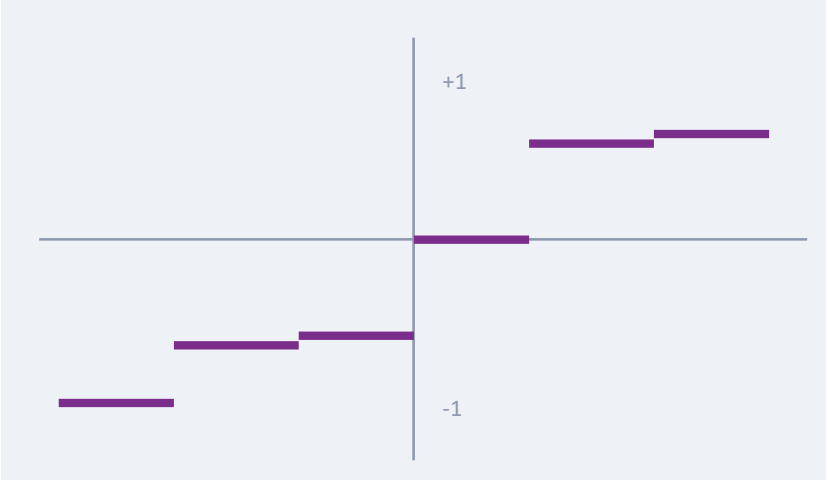
Sum = 1.0 ✓

Key Rules

- Always used LAST layer for multi-class
- Output = probability for each class
- All outputs sum to exactly 1.0
- Biggest logit → highest probability
- Paired with Cross-Entropy loss
- Never use in hidden layers

02 / Tanh & Comparison Summary

$\tanh(x) = (e^x - e^{-x}) / (e^x + e^{-x}) \rightarrow$ output $\in (-1, +1)$



tanh is zero-centered — better than Sigmoid for hidden layers!

02 / Tanh & Comparison Summary

Activation Function Cheat Sheet

Function	Output Range	Best For	Problem
ReLU	$(0, +\infty)$	Hidden layers — default choice	Dying ReLU
Sigmoid	$(0, 1)$	Binary output layer	Vanishing gradient

03 / The Forward Pass — Step by Step

We'll classify a handwritten digit image from scratch

1 Input: Flatten the Image

28×28 pixel image = 784 numbers

Each pixel value: 0 (black) to 255 (white)

Normalize: divide by 255 → values 0.0 to 1.0

Input vector x has shape [784]

2 Hidden Layer 1 (128 neurons)

$z^1 = W^1x + b^1$

W^1 shape: [128, 784] (128 × 784 = 100,352 weights!)

b^1 shape: [128]

$a^1 = \text{ReLU}(z^1)$ → 128 values, all ≥ 0

3 Hidden Layer 2 (64 neurons)

$z^2 = W^2a^1 + b^2$

W^2 shape: [64, 128]

b^2 shape: [64]

$a^2 = \text{ReLU}(z^2)$ → 64 values

4 Output Layer (10 neurons)

$z^3 = W^3a^2 + b^3$

W^3 shape: [10, 64]

b^3 shape: [10]

$\hat{y} = \text{Softmax}(z^3)$ → 10 probabilities (one per digit)

04 / Loss Functions — Measuring Mistakes

Cross-Entropy Loss (Classification)

$L = -\Sigma\ y_true \times \log(\hat{y}_predicted)$ → Penalizes wrong confident predictions HEAVILY

Intuition with Numbers:

True label	Model output (Softmax)	Loss	Meaning
Digit 3 (correct)	P(3)=0.95, P(7)=0.05	−log(0.95) = 0.05	Very small loss
Digit 3 (correct)	P(3)=0.51, P(7)=0.49	−log(0.51) = 0.67	Medium loss
Digit 3 (correct)	P(3)=0.05, P(7)=0.90	−log(0.05) = 3.00	Large loss

MSE Loss (Regression)

$L = (1/n)\ \Sigma\ (y - \hat{y})^2$

For continuous outputs (predict house price, temperature)

The goal of training: minimize the loss!

Optimizer adjusts weights to find the lowest point of the loss landscape — like rolling a ball downhill.

04 / Backpropagation — How the Network Learns

The chain rule of calculus, applied layer by layer

1

Compute the Loss

$L = \text{CrossEntropy}(\hat{y}, y_true)$

Compare prediction to ground truth. Get a number that measures how wrong we are.

2

Gradient at Output

$\partial L/\partial \hat{y} = \hat{y} - y_true$ (for cross-entropy)

How much did each output neuron contribute to the error?

3

Chain Rule — Layer by Layer

$\partial L/\partial W^2 = \partial L/\partial a^2 \times \partial a^2/\partial z^2 \times \partial z^2/\partial W^2$

Multiply gradients backwards through the network using the chain rule.

4

Update Weights (Gradient Descent)

$W \leftarrow W - \eta \times \partial L/\partial W$

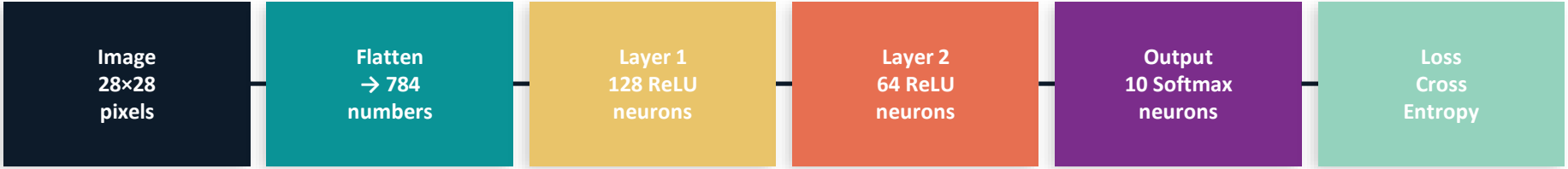
Move every weight slightly in the direction that reduces the loss. η = learning rate.

Repeat forward pass + backprop for every batch × every epoch until loss is minimized.

05 / Full Training Example: Digit Classifier

The Complete Picture — One Training Step

Input: image of digit '3' | True label: class index 3



Detailed Values — Following One Sample Through the Network:

Input	$x = [0.0, 0.12, 0.87, 0.95, 0.78, \ldots\ 0.0]$ shape: [784]
Layer 1 (partial)	$z^1[0] = 0.23\times0.0 + 0.87\times0.12 + (-0.34)\times0.87 + \ldots + b^1[0] = 0.42$ $a^1[0] = \mathrm{ReLU}(0.42) = 0.42$ ← positive, passes through
Layer 1 (partial)	$z^1[5] = \ldots = -0.71$ $a^1[5] = \mathrm{ReLU}(-0.71) = 0.0$ ← negative, blocked (dead for this input)
Output (Softmax)	$z^3 = [-1.2,\ 0.3,\ 0.8,\ 3.1,\ -0.4,\ 0.1, \ldots]$ $\hat{y} = [0.01, 0.04, 0.07,\ 0.85, 0.02,\ \ldots]$ ← class 3 wins with 85%
Loss	$L = -\log(\hat{y}[3]) = -\log(0.85) = 0.163$ ← small, model is mostly right
After 100 epochs	Loss drops from ~2.3 → ~0.08 | Accuracy: 10% → 97%

05 / PyTorch Code: The Full Example

Real code — from image to prediction, training loop included

```
import torch, torch.nn as nn
# 1. Define the Model
class DigitNet(nn.Module):
    def __init__(self):
        super().__init__()
        self.fc1 = nn.Linear(784, 128)  # Layer 1: 784→128
        self.fc2 = nn.Linear(128, 64)   # Layer 2: 128→64
        self.fc3 = nn.Linear(64, 10)    # Output:  64→10
    def forward(self, x):
        x = x.view(-1, 784)             # Flatten image
        x = F.relu(self.fc1(x))         # Layer 1 + ReLU
        x = F.relu(self.fc2(x))         # Layer 2 + ReLU
        return self.fc3(x)              # Raw logits (Softmax inside loss)

# 2. Train
model = DigitNet()
optimizer = torch.optim.Adam(model.parameters(), lr=0.001)
criterion = nn.CrossEntropyLoss()  # Applies Softmax internally

for epoch in range(10):
    pred = model(images)             # Forward pass
    loss = criterion(pred, labels)   # Compute loss
    optimizer.zero_grad()            # Clear old gradients
    loss.backward()                  # Backpropagation
    optimizer.step()                 # Update weights
```

That's the entire loop: Forward → Loss → Backprop → Update. Repeat until accuracy is high.

Key Takeaways

Weights

Numbers that multiply inputs — the memory of the network. Initialized randomly, updated by backprop.

Activation Functions

Introduce non-linearity. ReLU for hidden layers, Sigmoid for binary output, Softmax for multi-class output.

Forward Pass

Data flows: Input → weighted sums → activations → output. One matrix multiply + activation per layer.

Loss Function

Measures the mistake. Cross-entropy for classification. Minimizing loss IS the goal of training.

Backpropagation

Chain rule flowing backwards. Computes how much each weight contributed to the error.

Training Loop

Forward → Loss → Backward → Update. Repeat for many epochs until loss is low and accuracy is high.

"Understanding what happens inside each layer is what separates a user from a builder."